

Smart Mirror

Versión Vibecoding IA

Lasorsa, Lautaro ; Licarzi, Florencia Berenice; Abaca, Ivan ; Gonzalez, Luca ; Morales, Felipe

42394430, 43458509, 43262563, 43895910, 40853807

Lunes, 1

Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Contenido

Introducción	2
Herramienta de IA Utilizado	3
Modelo de IA utilizado	3
Prompts Técnicos Utilizados.....	3
Prompt Técnico 1- App de control de Espejo Inteligente (IoT).....	3
LENGUAJE DE PROGRAMACIÓN	3
PLATAFORMA	3
FUNCIONALIDAD A IMPLEMENTAR	3
REQUISITOS ESPECÍFICOS	3
RESTRICCIONES	5
Prompt Técnico 2 – Rediseño de Fragment de Sensores:	6
LENGUAJE:	6
PLATAFORMA:	6
FUNCIONALIDAD A IMPLEMENTAR:	6
Cantidad de Tokens Utilizados	7
Tiempo para obtener una versión funcional.....	7
Rúbrica de Funcionalidad.....	8
Conclusión	8

Introducción

El presente informe se enmarca en el trabajo práctico de la asignatura, cuyo objetivo es analizar y documentar el proceso de desarrollo de una aplicación móvil mediante la asistencia de herramientas de inteligencia artificial, práctica conocida como “vibecoding”. Se evalúa haciendo la App para Smart Mirror, un sistema de espejo inteligente basado en IoT que integra sensores de distancia y de luz, controlado desde una aplicación Android que se comunica con el hardware embebido a través del protocolo MQTT.

A lo largo de este documento se detallan la herramienta y el modelo de inteligencia artificial utilizados, los prompts técnicos empleados para guiar la generación de código, así como las métricas relevadas en cuanto a tiempo de desarrollo, consumo de recursos y cumplimiento de los requisitos funcionales establecidos. El propósito es evidenciar de manera concreta las diferencias entre un desarrollo tradicional y uno asistido por IA, tanto en términos de productividad como de calidad del resultado obtenido.

Herramienta de IA Utilizado

La herramienta de inteligencia artificial utilizada para el desarrollo del proyecto fue Claude Code, un entorno de trabajo asistido por IA integrado en la terminal que permite generar, editar y depurar código de forma interactiva a lo largo de todo el ciclo de desarrollo.

Modelo de IA utilizado

El modelo de inteligencia artificial empleado fue Claude Sonnet 5, seleccionado por su capacidad para interpretar consignas técnicas detalladas y generar código funcional manera precisa y eficiente.

Prompts Técnicos Utilizados

Prompt Técnico 1- App de control de Espejo Inteligente (IoT)

LENGUAJE DE PROGRAMACIÓN

- Java.
- Android Studio como entorno de desarrollo.
- UI con XML Views (Activities y Fragments).
- Código simple y modular: una clase por pantalla, sin mezclar lógica de conexión con lógica de UI.

PLATAFORMA

- App nativa para Android, minSdk 26 o superior.
- Debe correr en emulador y en dispositivo físico.
- Se conecta a un espejo IoT a través de un broker **MQTT** (protocolo `tcp://`), cuya IP y puerto ingresa el usuario manualmente. No requiere backend propio: la comunicación es directa app ↔ broker MQTT ↔ hardware.
- Incluir un modo de datos simulados (activable por el usuario) que genere valores de telemetría ficticios, para poder probar toda la app sin tener el hardware ni un broker real disponibles.

FUNCIONALIDAD A IMPLEMENTAR

Una app para controlar un espejo inteligente que:

1. Muestra en tiempo real la telemetría de dos sensores de distancia y un sensor de luz.
2. Permite un modo manual para regular la posición del espejo y la intensidad de una luz.
3. Permite encender/apagar la luz agitando el celular (detección de "shake" con el acelerómetro).
4. Muestra un historial gráfico de los valores de telemetría recibidos durante la sesión.
5. Alerta al usuario si permanece demasiado tiempo parado frente al espejo.

REQUISITOS ESPECÍFICOS

Comunicación MQTT

- Cliente MQTT: Eclipse Paho para Android (`org.eclipse.paho:org.eclipse.paho.client.mqttv3`).
- Tópicos de suscripción (telemetría entrante):
 - o ``espejo/sensor/distancia1``
 - o ``espejo/sensor/distancia2``
 - o ``espejo/sensor/luz``
- Tópicos de publicación (comandos salientes):
 - o ``espejo/control/modo`` → ``"manual"`` / ``"automatico"``
 - o ``espejo/control/servo`` → entero 0–180
 - o ``espejo/control/luz`` → entero 0–255
 - o ``espejo/control/led`` → ``"1"`` / ``"0"``
- Payloads en texto plano (no JSON).
- Los mensajes MQTT entrantes deben propagarse a las pantallas que los necesitan (por ejemplo con un broadcast local o un listener central), no leerse directamente desde cada pantalla.

Pantalla 1 — Bienvenida

- Título de bienvenida, imagen ilustrativa, texto instructivo y botón "Comenzar".
- Al tocar "Comenzar": pedir el permiso de notificaciones (Android 13+) si no fue otorgado, y navegar a la pantalla de Conexión.

Pantalla 2 — Conexión

- Campo de texto para IP, campo numérico para puerto, botón "Conectar", texto de estado, y un toggle "Usar datos simulados".
- Al presionar "Conectar" (en un hilo de background, sin bloquear la UI):
- Si faltan IP o puerto (y no está activado el modo simulado): mostrar "Completa todos los campos".
- Mientras conecta: mostrar "Conectando...".
- Si conecta con éxito (o si el modo simulado está activo): navegar al Dashboard.
- Si falla: mostrar "No se pudo conectar" y permanecer en la pantalla.

Pantalla 3 — Dashboard

- Contenedor con una barra de navegación inferior de 3 secciones: **Sensores**, **Control**, **Historial**.
- Al entrar, suscribirse a los 3 tópicos de sensores y arrancar un servicio en segundo plano para la alerta de tiempo de uso (ver más abajo).
- Si se pierde la conexión MQTT, avisar al usuario (Toast o Snackbar).

Sección Sensores

- 3 tarjetas: "Distancia 1", "Distancia 2" y "Luz", cada una con el valor recibido en tipografía grande.
- Antes de recibir el primer dato de cada sensor, mostrar un placeholder (por ejemplo "-- cm", "-- luz").
- Pantalla de solo lectura, sin acciones del usuario.

Sección Control

- Switch "Manual": activa/desactiva el modo manual y publica el tópico ``espejo/control/modo``.

- Slider de posición (0–180, valor inicial 90 = centro), con texto que indique si está a la izquierda, al centro o a la derecha del punto medio.
- Slider de intensidad de luz (0–255), con texto que muestre el valor numérico.
- Botón "Enviar", habilitado solo si el modo manual está activo: al presionarlo publica los valores actuales de ambos sliders (`espejo/control/servo` y `espejo/control/luz`). Los sliders no envían nada mientras se arrastran, solo al confirmar con el botón.
- Switch "Shake": solo puede activarse si el modo manual está activo (si no, revertirlo y avisar "Activar modo manual primero").
- Al activarse, escuchar el acelerómetro (`TYPE\_ACCELEROMETER`) y calcular la magnitud del vector de aceleración ($\sqrt{x^2+y^2+z^2}$).
- Si supera el umbral ****12****, y pasó más de ****1000 ms**** desde la última detección (para evitar múltiples disparos por una sola sacudida), alternar un estado de encendido/apagado y publicar `"1"/"0"` en `espejo/control/led`.
- Dejar de escuchar el sensor al salir de la pantalla o al desactivar el switch.

Sección Historial

- 3 gráficos de líneas (uno por sensor: distancia1, distancia2, luz), usando una librería de gráficos para Android (por ejemplo MPAndroidChart).
- Cada valor nuevo recibido por MQTT se agrega como un punto al gráfico correspondiente. Los datos se guardan solo en memoria durante la sesión, sin base de datos.
- Si todavía no llegó ningún dato de un sensor, el gráfico correspondiente queda vacío.

Service - Alerta de tiempo de uso

- Un servicio en primer plano (foreground service, con notificación persistente de baja prioridad) que:
 - Escucha los valores de los sensores de distancia.
 - Si la distancia es menor a un umbral (por ejemplo 390, en la misma unidad que reporta el sensor) de forma continua durante más de 5 segundos, dispara una notificación de alta prioridad avisando que se llevó mucho tiempo frente al espejo.
 - Resetea el conteo apenas la distancia vuelve a superar el umbral.
 - Se detiene al salir del Dashboard.

RESTRICCIONES

- No usar datos reales sensibles (IPs o credenciales reales) como valores por defecto en el código.
- No agregar funcionalidades fuera de lo descrito acá (por ejemplo: sin login de usuario, sin backend en la nube, sin control de múltiples dispositivos), salvo el modo de datos simulados.
- No concentrar toda la lógica en una sola clase: separar claramente la comunicación MQTT, la lógica de cada pantalla y la UI.
- No dejar ninguna de las pantallas o secciones incompleta.

- No bloquear el hilo principal con operaciones de red; usar hilos de background o `AsyncTask`/`Thread` para conectar y publicar por MQTT.
 - No agregar librerías o dependencias que no sean necesarias para cumplir los requisitos de este documento.
-

Prompt Técnico 2 – Rediseño de Fragment de Sensores:

LENGUAJE:

- UI con XML Views (Activities y Fragments).

PLATAFORMA:

- App de Android

FUNCIONALIDAD A IMPLEMENTAR:

Modificar la pantalla principal de monitoreo de sensores del proyecto “Espejo Inteligente”. Rediseñar la pantalla donde actualmente se muestran las mediciones de los sensores, para que sea más visual y clara para el usuario.

REQUISITOS ESPECIFICOS:

En la parte superior de la pantalla debe mostrarse una imagen, ilustración o dibujo representativo del espejo inteligente. Esta imagen debe ocupar un espacio destacado, estar centrada y tener un estilo limpio, moderno y coherente con una aplicación IoT.

Debajo de la imagen del espejo deben mostrarse 3 mini cards informativas, una por cada sensor:

1. Sensor de distancia izquierdo

- Debe mostrar el nombre del sensor.
- Debe mostrar el valor medido actualmente.
- Unidad sugerida: cm.

2. Sensor de distancia derecho

- Debe mostrar el nombre del sensor.
- Debe mostrar el valor medido actualmente.
- Unidad sugerida: cm.

3. Sensor de luz ambiente

- Debe mostrar el nombre del sensor.
- Debe mostrar el valor medido actualmente.
- Unidad sugerida: lux, porcentaje o valor analógico, según cómo esté implementado actualmente.

Restricciones:

- No modificar la lógica de lectura de sensores.
- No modificar la lógica de comunicación con el ESP32.
- Reutilizar las variables o datos ya existentes que actualmente alimentan la pantalla.
- El cambio debe ser principalmente visual y de organización de la información.

- Las cards deben actualizarse dinámicamente cuando cambien los valores de los sensores.
- Mantener una interfaz simple, clara y responsive.

Cantidad de Tokens Utilizados

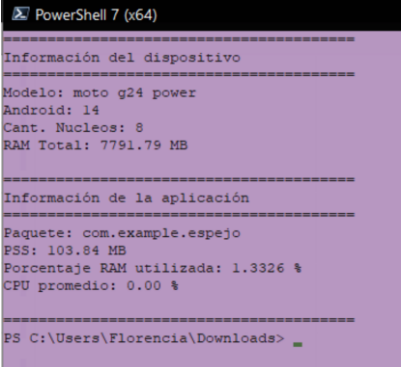
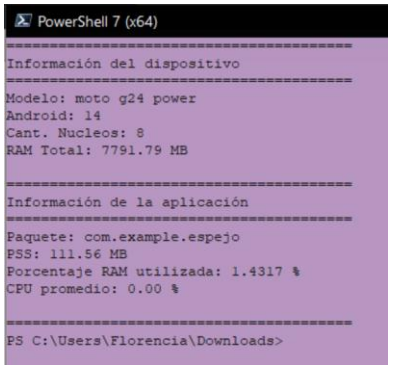
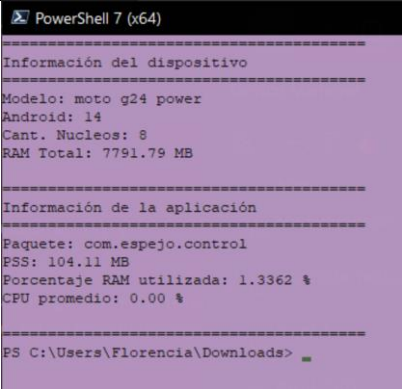
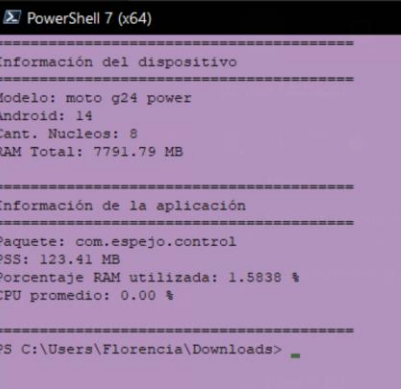
El desarrollo del proyecto insumió aproximadamente un 20% de la sesión (Current Session) correspondiente a Standard seat del plan “Team”, lo que resultó suficiente para completar ambos prompts técnicos sin necesidad de iniciar una nueva sesión.

Tiempo para obtener una versión funcional

A continuación, se compara el tiempo estimado de desarrollo manual con el tiempo real obtenido utilizando asistencia de inteligencia artificial:

Manual	IA
25 horas	3 horas

Métricas CPU y Memoria

Manual (Simple)	Manual (Shake)
	
IA (Simple)	IA (Shake)
	

Al comparar ambas apps, el consumo de memoria en reposo es prácticamente el mismo (~104 MB). Sin embargo, al detectar el shake, la app vibecodeada (com.espejo.control) sube bastante más de memoria (+19,30 MB) que la desarrollada por nosotros (com.example.espejo, +7,72 MB), lo que indica que maneja ese evento de forma menos eficiente.

Rúbrica de Funcionalidad

CRITERIOS	SUFICIENTEMENTE LOGRADO (A)	MEDIANAMENTE LOGRADO (B)	INSUFICIENTEMENTE LOGRADO (C)	Puntaje	Eval
FUNCIONA SIN ERRORES	Compila y funciona sin errores.	Funciona con errores menores o warnings.	Presenta errores que impiden su uso.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	17
UTILIZA EJECUCIÓN EN BACKGROUND	Implementa correctamente tareas en background.	Implementa background con algunas fallas.	No implementa tareas en background.	A: Hasta 10 pts. B: Hasta 5 pts. C: 0	5
UTILIZA SENSORES Y PERMISOS DE ANDROID	Utiliza correctamente sensores y permisos.	Presenta problemas menores en sensores o permisos.	Sensores o permisos no funcionan correctamente.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	20
COMUNICACIÓN CON EL EMBEBIDO	Comunicación estable en ambos sentidos.	Comunicación parcial o inestable.	La comunicación falla.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	20
CUMPLIMIENTO DEL OBJETIVO FUNCIONAL	Cumple todos los requisitos funcionales.	Cumple parcialmente los requisitos.	No cumple los requisitos mínimos.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	18
DISEÑO DE LA APP	Interfaz clara y fácil de usar.	Interfaz funcional con algunos problemas.	Interfaz deficiente o difícil de usar.	A: Hasta 10 pts. B: Hasta 5 pts. C: 0	8
TOTAL				100	88

Conclusión

La aplicación cumple de manera sólida con la mayoría de los criterios evaluados, destacándose especialmente en la comunicación con el embebido y en el uso correcto de sensores y permisos, junto con un funcionamiento estable y con pocos errores. El aspecto más débil es la ejecución de tareas en background, que presenta algunas fallas y representa el principal punto a mejorar. En conjunto, el resultado refleja una implementación funcional y prolija, con margen de optimización en ese aspecto puntual.